

Modeling infectious diseases using R

Practical Session:

Basic programming in R for the practical sessions

Ziv Shkedy, Rashider Aloni, Leyla Kodalci
Center for Statistics, Hasselt University, Belgium

First Training workshop, Cape Town, South Africa
12/10/2026-16/12/2026



Sustainable Training in Epidemiological Analysis and Modelling
for Data-Driven Decision Making in Africa

What do we cover in this practical session ?

- Basic programming in R.
- Working with objects.
- User functions.
- Working with functions.

R programs

- HTML file:
 - eR_prog0_STEAM3D_10_2026_V1.html.
- R program:
 - eR-STEAM3D_Introduction_V1_1-_2026_R.
- R markdown program:
 - eR_prog0_STEAM3D_10_2026_V1.Rmd.

Objects

- We define two objects x and y (=the data):

```
> x<-c(1.9,1.2,0.7,2.7,1.2,3.1,2.3,2.1,2.1,1.4)
> x
[1] 1.9 1.2 0.7 2.7 1.2 3.1 2.3 2.1 2.1 1.4
> y<-c(1.8,1.1,0.6,2.7,1.4,3.0,2.5,1.9,1.8,1.4)
> y
[1] 1.8 1.1 0.6 2.7 1.4 3.0 2.5 1.9 1.8 1.4
```

Calculate the mean

- We can calculate the sample mean for x and y using the `mean()` function in R.

`mean(data)`



R function to calculate
the sample mean.

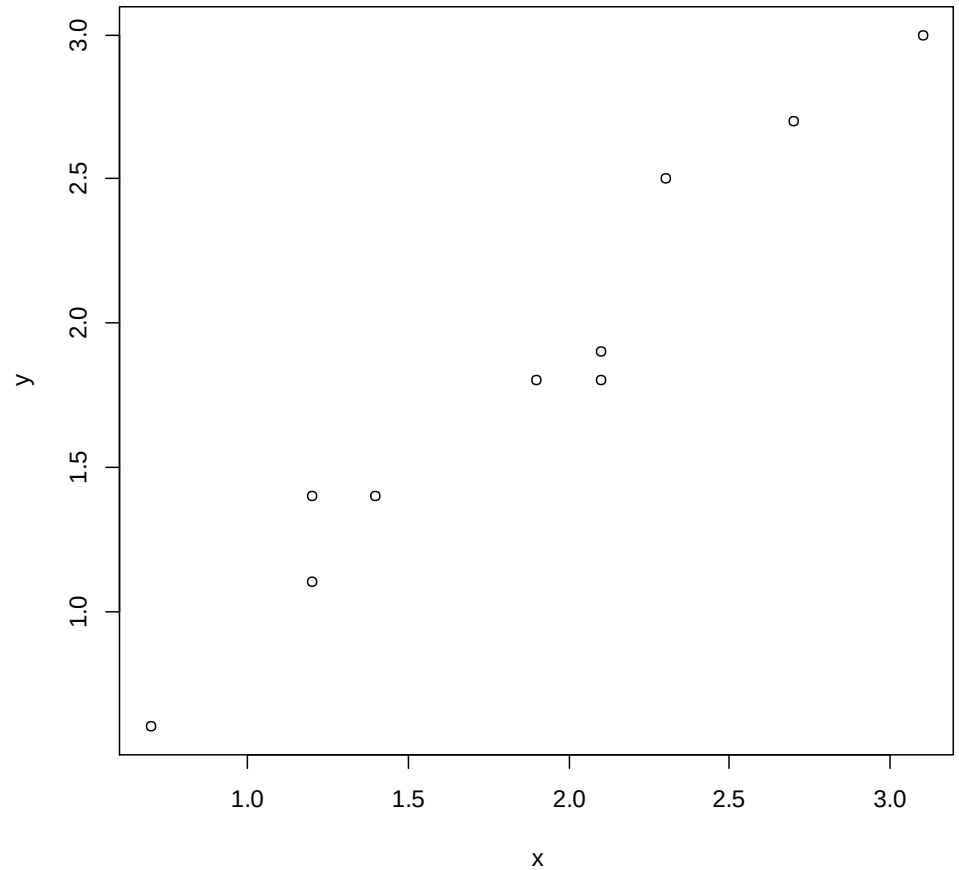
The data.

```
> mean(x)
[1] 1.87
> mean(y)
[1] 1.82
```

Correlation

```
> cor(x,y)  
[1] 0.9774492
```

↓
R function that calculate
the correlation.



The function `plot()`
produces a scatterplot.

```
> plot(x,y)
```

A user function: general form

- A user function is a function that we write to do something in R.

```
function name<-function(x)
```

```
{
```

R commands (what do you want that the function will do for you.....)

```
}
```

A user function: general form

```
function name<-function(x,y)
{  The function starts here

}  The function ends here
```

Input: x & y : the sample=the data

Output:

- Descriptive statistics: mean, correlation.
- Graphical output: scatterplot

R functions for the output:

```
mean ()
cor ()
plot ()
```

A user function: general form

- A small program to produce the output:

```
mean (x)  
mean (y)  
cor (x, y)  
plot (x, y)
```

See slides 5-6.



x & y: R objects with the data

A user function: example

The name of
the function



```
discript<-function(x,y)
```



```
{  
  print(mean(x))  
  print(mean(y))  
  print(cor(x,y))  
  plot(x,y)  
}
```



The input: R
objects with
the data



The function's output:
calculate the mean and x
and y, calculate the
correlation and produce
a plot.

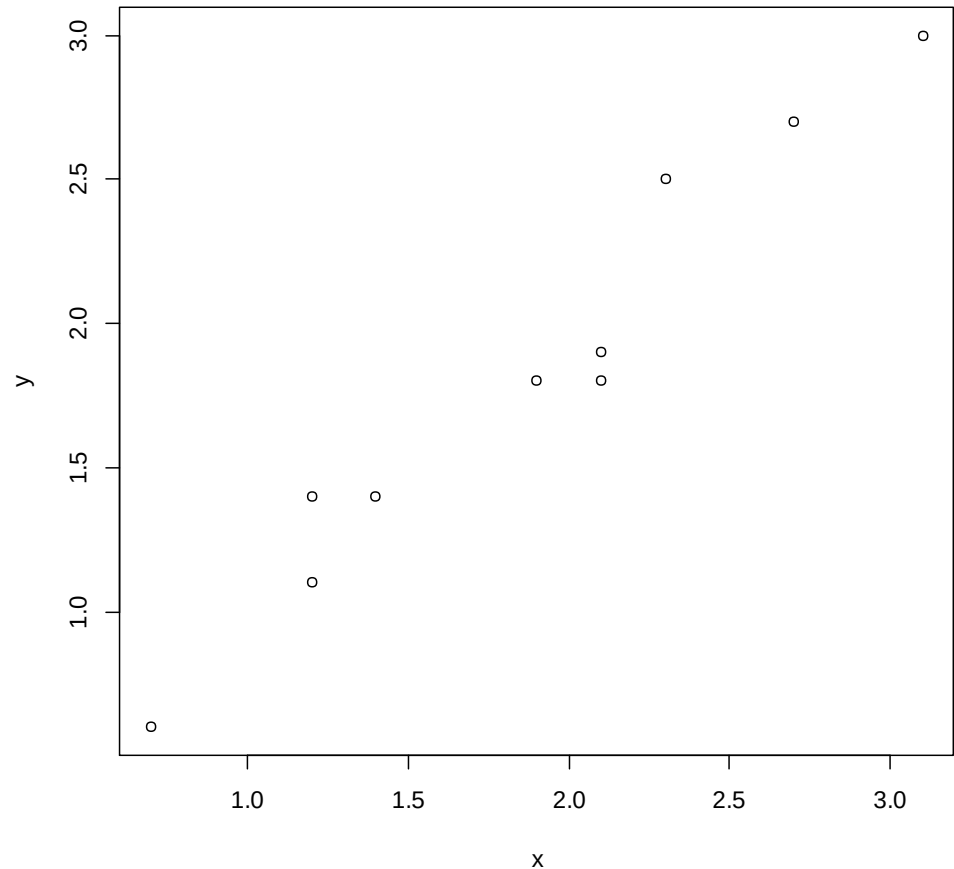


When you finish to write the function, you need to run it first in R.
Highlight the function code and run.

Applying the function to the data

```
> discript(x,y)
[1] 1.87
[1] 1.82
[1] 0.9774492
```

- Output:
 - The mean and x and y.
 - The correlation.
 - Scatterplot.



The cars Data set in R

1. Write **cars** in the script window.
2. Submit

```
> cars
```

```
   speed dist
1      4    2
2      4   10
3      7    4
4      7   22
5      8   16
6      9   10
.      .    .
.      .    .
48     24   93
49     24  120
50     25   85
```

Data frame in R

```
> help(cars)
```

```
>
```

Speed and Stopping Distances of Cars Description

The data give the speed of cars and the distances taken to stop. Note that the data were recorded in the 1920s.

[,1]	speed	numeric	Speed (mph)
[,2]	dist	numeric	Stopping distance (ft)

What do we want to do ?

- For the cars dataset:
 - Calculate the mean of the cars' speed.
 - Calculate the mean of the cars' stopping distance.
 - Calculate the correlation between cars' speed and stopping distance.
 - Produce a scatterplot.

Applying the function to the cars data

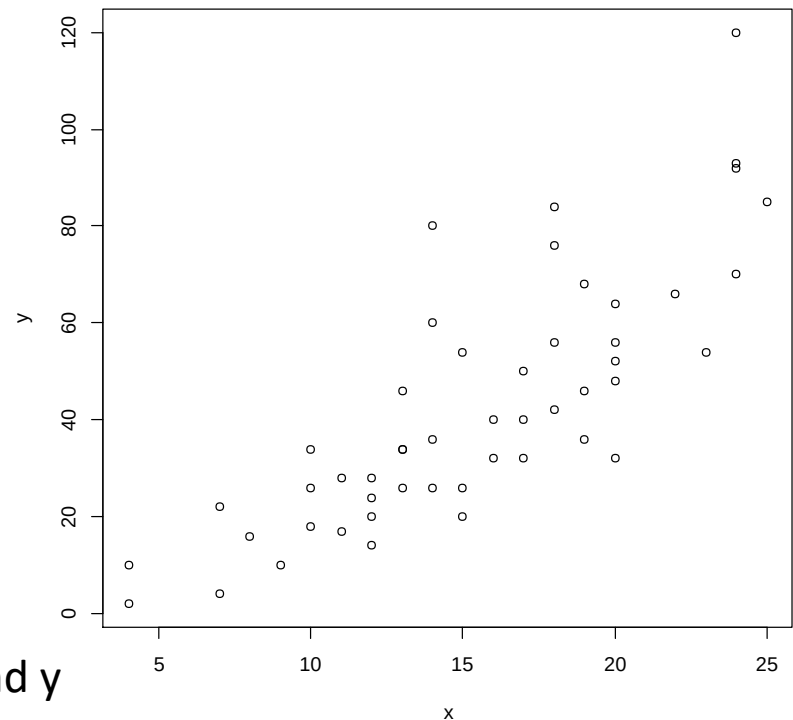
```
> head(cars)
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

The data in R

```
> discript(cars[,1],cars[,2])
[1] 15.4
[1] 42.98
[1] 0.8068949
```

- The function's output:
 - The mean of x.
 - The mean of y.
 - The correlation.
 - Scatterplot.

- The input for the function
- The data = the x and y variables.



How to produce the output in presented in
the slides

How to produce the output ?

- Open R/R Studio.
- Open the file:
 - [ModelingIDinR0_V1_Introduction_10_2026.R.](#)
- Run the code in the file.

The HTML file

The screenshot shows a web browser window with the following elements:

- Browser Tab:** er_prog0_STEAM3D_2026-V1.k
- Address Bar:** C:/Ziv_Temp_2023/Tarylee_STEAM-3D/Workshop1_12_10_26/ShortCourse/Practicalessions_New/er_prog0_STEAM3D_2026-V1.html
- Table of Contents (Left Sidebar):**
 - 1. Defining objects in R
 - 2. Calculate the mean of x and y in R
 - 3. Corelation between x and y
 - 4. User functions
 - 5. The cars dataset
- Main Content Area:**
 - Date:** 02-10-2026
 - Course:** >eR-BioStat and STAEM-3D
 - Title:** Modeling infectious diseases using R - basic programming in R
 - Authors:** Ziv Shkedy, Rashider Aloni and Leyla Kodalcı
 - Section 1: Defining objects in R**

We consider two vectors, x and y .

$x = (1.9, 1.2, 0.7, 2.7, 1.2, 3.1, 2.3, 2.1, 2.1, 1.4)$ and $y = (1.8, 1.1, 0.6, 2.7, 1.4, 3.0, 2.5, 1.9, 1.8, 1.4)$.

We define the two vectors in R in teh follwoing way. For x :

```
## [1] 1.9 1.2 0.7 2.7 1.2 3.1 2.3 2.1 2.1 1.4
```

and for y :

```
## [1] 1.8 1.1 0.6 2.7 1.4 3.0 2.5 1.9 1.8 1.4
```
 - Section 2: Calculate the mean of x and y in R**

Our first data analysis task is to calculate the mean of x and y . In R, the function `mean()` can be used to calculate the sample mean. In our example we use `mean(x)` and `mean(y)`:

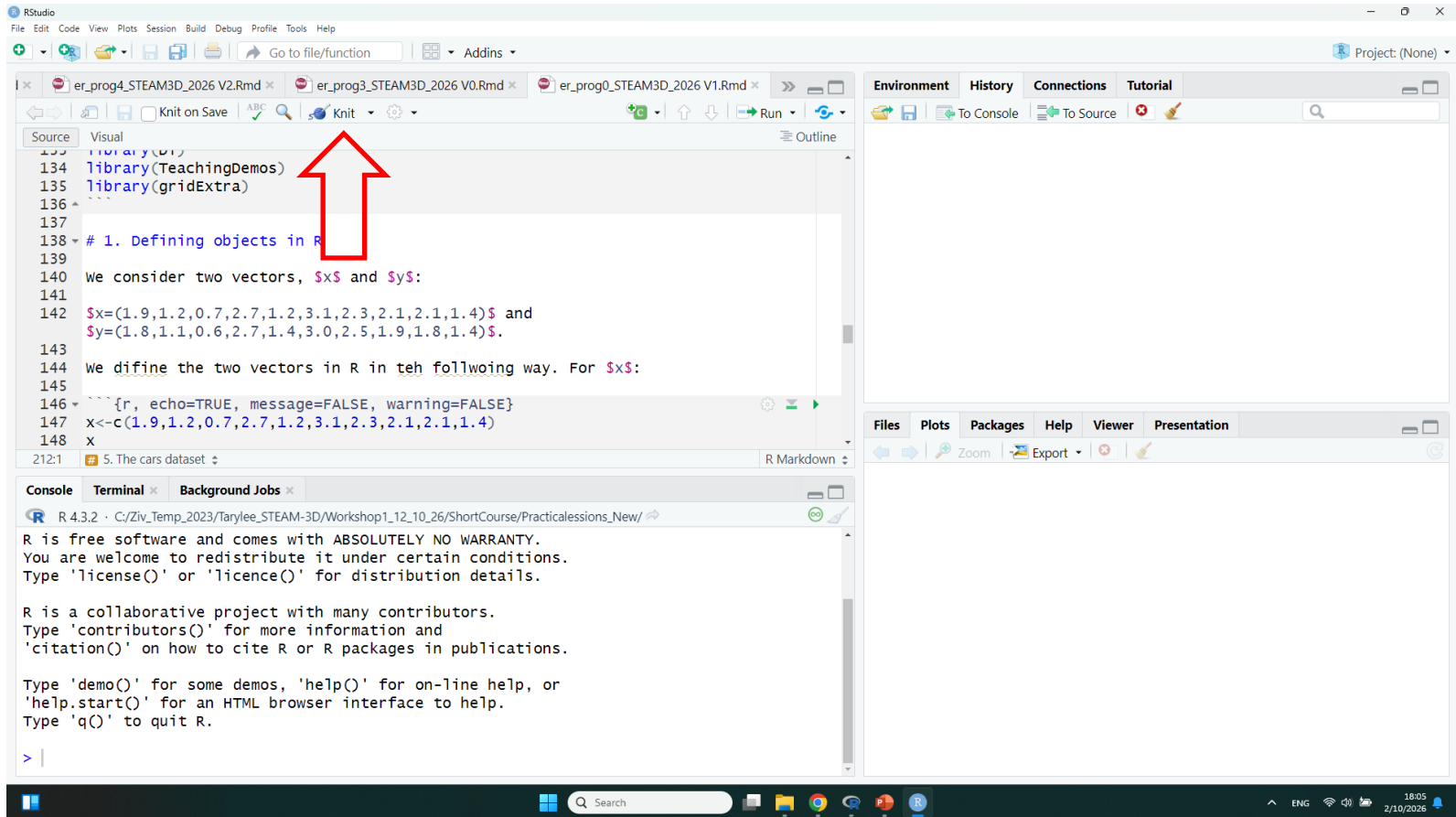
```
## [1] 1.87
```

The Windows taskbar at the bottom shows the Start button, Search bar, and several application icons (File Explorer, Chrome, Edge, PowerPoint, etc.). The system clock indicates 18:04 on 2/10/2026.

How to produce the HTML file ?

- Open R studio.
- Open the file:
 - eR_prog0_STEAM3D_10_2026_V1.Rmd.
- Knit the file.

How to produce the HTML file ?



The R package deSolve

The deSolve package

- We use the R package `deSolve` to run transmission models.
- In particular we use the R function `ode()` to run the model.

Running a transmission model using the R package `deSolve`

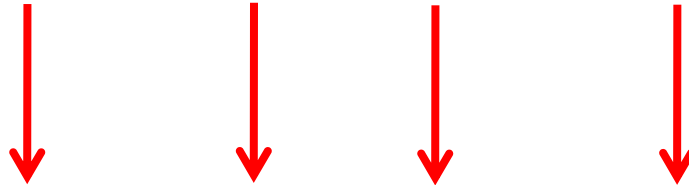
```
require(deSolve)  
out<-as.data.frame(ode(y=state,times=times,func=SIR,parms=parameters))
```



A function in R that can be used for Numerical integration using ordinary differential equation

Running a transmission model using the R package `deSolve`

```
require(deSolve)  
out<-as.data.frame(ode(y=state,times=times,func=SIR,parms=parameters))
```



- Vectors with input information about the model that we want to run.
- The input that we need to give to the function `ode ( )` .